

Erläuterung ausgewählter Softwariemetriken

1. Weighted Methods per Class (WMC)

Beschreibung

Die Metrik **Weighted Methods per Class (WMC)** beschreibt die Komplexität einer Klasse. Im CK-Werkzeug entspricht sie im Wesentlichen der Summe der zyklomatischen Komplexität aller Methoden einer Klasse.

Je höher der WMC-Wert ist,

- desto komplexer ist eine Klasse,
- desto schwieriger ist sie zu verstehen,
- desto aufwendiger werden Tests und Wartung.

Niedrige WMC-Werte sind daher grundsätzlich anzustreben.

Beispiel einer hohen WMC

Eine Methode mit vielen Bedingungen und Verzweigungen erhöht den WMC-Wert.

```
public void handleInput(String input) {  
    if (input == null) {  
        return;  
    }  
  
    if (input.matches(COMPLEX_REGEX)) {  
        ...  
    } else if (...) {  
        ...  
    } else if (...) {  
        ...  
    } else {  
        ...  
    }  
}
```

Hier entstehen mehrere Entscheidungswege, wodurch die Komplexität der Methode steigt.

Verbesserte Version

Durch das Auslagern des regulären Ausdrucks in eine eigene Methode wird die eigentliche Methode übersichtlicher.

```
public void handleInput(String input) {
```

```

        if (input == null) {
            return;
        }

        if (matchesPattern(input)) {
            ...
        } else if (...) {
            ...
        }
    }

private boolean matchesPattern(String input) {
    return input.matches(COMPLEX_REGEX);
}

```

Obwohl sich der WMC-Wert nur geringfügig verändert, verbessert sich die Lesbarkeit erheblich. Die Methode besitzt nun eine klar erkennbare Aufgabe und folgt dem Clean-Code-Prinzip der geringen Verantwortlichkeit (Single Responsibility Principle).

2. Coupling Between Objects (CBO)

Beschreibung

Die Metrik **Coupling Between Objects (CBO)** beschreibt, mit wie vielen anderen Klassen eine Klasse direkt gekoppelt ist.

Eine hohe Kopplung führt dazu, dass Änderungen an einer Klasse häufig Auswirkungen auf weitere Klassen haben.

Ein niedriger CBO-Wert verbessert

- Wartbarkeit,
 - Testbarkeit,
 - Wiederverwendbarkeit.
-

Beispiel einer hohen Kopplung

```

public class Game {

    private Player player;
    private Enemy enemy;
    private Inventory inventory;
    private SoundManager soundManager;
    private GameWindow window;
    private SaveManager saveManager;

    ...
}

```

Diese Klasse kennt viele andere Klassen direkt. Änderungen an einer dieser Klassen können Auswirkungen auf die Klasse `Game` haben.

Verbesserungsmöglichkeiten

Eine Möglichkeit besteht darin, Verantwortlichkeiten aufzuteilen.

```
public class Game {  
  
    private Player player;  
    private GameWorld world;  
  
}
```

Die Klasse `GameWorld` übernimmt anschließend die Verwaltung weiterer Komponenten.

Dadurch wird die direkte Kopplung reduziert und die Klasse übersichtlicher.

Warum nicht jede schlechte Metrik verbessert werden muss

Softwaremetriken liefern Hinweise auf mögliche Verbesserungen, stellen jedoch keine absoluten Qualitätskriterien dar.

Einige Klassen weisen aufgrund ihrer fachlichen Verantwortung zwangsläufig höhere Metrikwerte auf.

Beispiele sind:

- zentrale Spielsteuerung (`Game`)
- Hauptmenüs
- Levelverwaltung
- Server-Controller

Diese Klassen koordinieren bewusst viele Komponenten des Systems und besitzen deshalb naturgemäß einen höheren CBO-Wert.

Ebenso kann eine Klasse aufgrund umfangreicher Spiellogik einen erhöhten WMC-Wert besitzen. Würde diese Logik künstlich auf zahlreiche kleine Klassen verteilt, könnte dies die Architektur unnötig verkomplizieren und die Verständlichkeit verschlechtern.

Aus diesem Grund wurden ausschließlich Änderungen vorgenommen, die einen tatsächlichen Mehrwert hinsichtlich Lesbarkeit und Wartbarkeit bieten. Das durchgeführte Clean-Code-Refactoring verbessert die Verständlichkeit des Codes, ohne dessen Architektur unnötig zu verändern. Dieses Vorgehen entspricht dem Grundsatz, dass Softwaremetriken als Orientierung dienen und nicht isoliert optimiert werden sollten.

Zusammenfassend gab es keine Änderungen, welche signifikante Auswirkungen auf die Metriken hatten. Dies ist jedoch nach ausführlicher Betrachtung des ausgehenden Codes

entschieden worden. Entsprechend sind die obigen Codebeispiele lediglich beispielhaft dafür, wie solche Änderungen hätten aussehen können, ließen sich jedoch nicht derartig auf unser Projekt anwenden.